

1pf_threadsCreateFiles: Report

Inputs:

For the input process, the program uses the following function:

```
private static int pedir(Scanner sc, String texto, int minimo) {  
    while (true) {  
        System.out.print(texto);  
        if (sc.hasNextInt()) {  
            int valor = sc.nextInt();  
            if (valor >= minimo) {  
                return valor;  
            }  
            System.out.println("Tiene que ser >= " + minimo);  
        } else {  
            System.out.println(x: "Eso no es un número.");  
            sc.next();  
        }  
    }  
}
```

This allows us to keep asking until valid values are received. Under the main function, they are received with this fragment:

```
public static void main(String[] args) throws InterruptedException {  
    Scanner sc = new Scanner(System.in);  
  
    int n = pedir(sc, texto: "¿Cuántos threads? ", minimo: 1);  
    int m = pedir(sc, texto: "¿Cuántos mensajes máximo? ", minimo: 1);  
    int t = pedir(sc, texto: "¿Cuánto tiempo dormir máximo (ms)? ", minimo: 10);  
}
```

```
¿Cuántos threads? a  
Eso no es un número.  
¿Cuántos threads? 3  
¿Cuántos mensajes máximo? 10  
¿Cuánto tiempo dormir máximo (ms)? 20000
```

Process:

Each worker is an instance from the class “Formula”, which are initialized using the constructor that uses the input values:

```
public Formula(int id, int maxMensajes, int maxDormir) {  
    this.id = id;  
    this.maxMensajes = maxMensajes;  
    this.maxDormir = maxDormir;  
}
```

The rest of the processes have an assigned formula, one made to create the file, other to write in it and another to calculate the P used the random values.

```
private int calculateP(int c, int h, int a) {  
    return (c + h) * a;  
}  
  
/** Crea el archivo del hilo y abre el escritor. */  
private void createFile() throws IOException {  
    this.archivo = new File("formula_" + id + "_" + rand(min: 1, max: 10000) + ".txt");  
    this.archivo.createNewFile();  
    this.escriptor = new BufferedWriter(new FileWriter(archivo));  
}  
  
/** Escribe una línea en el archivo del hilo. */  
private void write(String mensaje) throws IOException {  
    escriptor.write(mensaje);  
    escriptor.newLine();  
    escriptor.flush(); // para poder ver el archivo mientras corre  
}
```

These are operated on the “run” function, as each worker is a thread itself, as well as using the “rand” function to assign the random value. In the execution process, the other requirements are fulfilled as the random “m” for iterations and the “t” for sleep timeout:

```
/** Aleatorio entre min y max, ambos incluidos. */  
private static int rand(int min, int max) {  
    return ThreadLocalRandom.current().nextInt(min, max + 1);  
}
```

```
@Override
public void run() {
    try {
        createFile();

        int m = rand(min: 1, max: mensajes);
        write("=== Hilo " + id + " | " + m + " mensajes ===");

        for (int i = 1; i <= m; i++) {
            int c = rand(min: 1, max: 100);
            int h = rand(min: 1, max: 100);
            int a = rand(min: 1, max: 100);
            int p = calculateP(c, h, a);
            this.P += p;

            write(String.format(format: "[%d/%d] P = (%d + %d) * %d = %d", i, m, c, h, a, p));

            int t = rand(min: 10, max: dormir);
            Thread.sleep(t);
        }
    }
}
```

In this process it is visible too that the corresponding C, H, and A (used to calculate P) are random as well as the P calculation uses the correct formula:

```
private int calculateP(int c, int h, int a) {
    return (c + h) * a;
}
```

At the end, each process calculates its own average P and determines how “good” it is.

```
double averageP = this.P/m;
String apreciacion;

System.out.println("Hilo " + id + " terminó -> " + archivo.getName());
System.out.println("Promedio de P: " + averageP);

if(averageP < 5000){
    apreciacion = "mala";
} else if(averageP > 15000){
    apreciacion = "buena";
} else{
    apreciacion = "promedio";
}
```

Output:

As required, we can open and see how the different iterations are written into the file:

```
formula_1_1279.txt
1  === Hilo 1 | 7 mensajes ===
2  [1/7] P = (32 + 29) * 88 = 5368
3
```

```
formula_1_1279.txt
1  === Hilo 1 | 7 mensajes ===
2  [1/7] P = (32 + 29) * 88 = 5368
3  [2/7] P = (8 + 69) * 100 = 7700
4
```

```
formula_1_1279.txt
1  === Hilo 1 | 7 mensajes ===
2  [1/7] P = (32 + 29) * 88 = 5368
3  [2/7] P = (8 + 69) * 100 = 7700
4  [3/7] P = (98 + 38) * 77 = 10472
5
```

Also, the watcher will print the state of each thread every 200 milliseconds, and after every thread has finished it stops the program, as there are no more threads to be watched.

```
for(Thread hilo : threads){
    String state = hilo.getState().toString();
    String name = hilo.getName();
    System.out.println("Thread: " + name + "; State: " + state);

    if (hilo.getState() != Thread.State.TERMINATED) {
        terminado = false;
    }
}

if(terminado){
    System.out.println(x: "Todos los hilos terminados, cerrando watcher");
    break;
}
Thread.sleep(millis: 200);
```

```
List<Thread> hilos = new ArrayList<>();
for (int i = 1; i <= n; i++) {
    Thread hilo = new Thread(new Formula(i, m, t), "formula-" + i);
    hilos.add(hilo);
    hilo.start();
}

Watcher w = new Watcher(hilos);
w.start();
```

```
Thread: formula-1; State: TIMED_WAITING
Thread: formula-2; State: TIMED_WAITING
Thread: formula-3; State: TERMINATED
Thread: formula-1; State: TIMED_WAITING
Thread: formula-2; State: TIMED_WAITING
Thread: formula-3; State: TERMINATED
Thread: formula-1; State: TIMED_WAITING
Thread: formula-2; State: TIMED_WAITING
Thread: formula-3; State: TERMINATED
```

```
Thread: formula-1; State: TIMED_WAITING
Thread: formula-2; State: TIMED_WAITING
Thread: formula-3; State: TERMINATED
Hilo 2 terminó -> formula_2_9558.txt
Promedio de P: 2991.8571428571427
Thread: formula-1; State: TIMED_WAITING
Apreciación: mala.
Thread: formula-2; State: RUNNABLE
Thread: formula-3; State: TERMINATED
Thread: formula-1; State: TIMED_WAITING
Thread: formula-2; State: TERMINATED
Thread: formula-3; State: TERMINATED
```

```
Listo: 3 archivos generados.
Thread: formula-1; State: TERMINATED
Thread: formula-2; State: TERMINATED
Thread: formula-3; State: TERMINATED
Todos los hilos terminados, cerrando watcher
```

After being terminated, each thread prints its average P and gives an evaluation (in this case, Thread 2 finished before thread 1:

```
Hilo 2 terminó -> formula_2_9558.txt
Promedio de P: 2991.8571428571427
Thread: formula-1; State: TIMED_WAITING
Apreciación: mala.
```

```
Hilo 1 terminó -> formula_1_1279.txt  
Promedio de P: 5609.142857142857  
Apreciación: promedio.
```

Requirements:

1. **Threads:** threads are used in several parts of the program, being that every worker that creates its own file is a thread itself and uses the “run” function. As well, the watcher that checks the threads is a different thread with its own “run” function. As threads, both require to be “started”.

```
public class Formula implements Runnable {  
  
    private final int id;  
    private final int maxMensajes;    // M  
    private final int maxDormir;      // T  
    public double P = 0;
```

```
public class Watcher extends Thread{  
    private List<Thread> threads;  
  
    Watcher(List<Thread> ts){  
        this.threads = ts;  
    }  
  
    @Override  
    public void run(){
```

```
List<Thread> hilos = new ArrayList<>();  
for (int i = 1; i <= n; i++) {  
    Thread hilo = new Thread(new Formula(i, m, t), "formula-" + i);  
    hilos.add(hilo);  
    hilo.start();  
}  
  
Watcher w = new Watcher(hilos);  
w.start();
```

2. **Sleep:** Both thread classes implement a sleep in their respective “run” functions. In the case of the “Formula” it has a sleep after writing in each file and the watcher has a sleep after each console print.

```
@Override
public void run() {
    try {
        createFile();

        int m = rand(min: 1, max: mensajes);
        write("=== Hilo " + id + " | " + m + " mensajes ===");

        for (int i = 1; i <= m; i++) {
            int c = rand(min: 1, max: 100);
            int h = rand(min: 1, max: 100);
            int a = rand(min: 1, max: 100);
            int p = calculateP(c, h, a);
            this.P += p;

            write(String.format(format: "[%d/%d] P = (%d + %d) * %d = %d", i, m, c, h, a, p));

            int t = rand(min: 10, max: dormir);
            Thread.sleep(t);
        }
    }
}
```

```
@Override
public void run(){
    while (true) {
        try {
            boolean terminado = true;

            for(Thread hilo : threads){
                String state = hilo.getState().toString();
                String name = hilo.getName();
                System.out.println("Thread: " + name + "; State: " + state);

                if (hilo.getState() != Thread.State.TERMINATED) {
                    terminado = false;
                }
            }

            if(terminado){
                System.out.println(x: "Todos los hilos terminados, cerrando watcher");
                break;
            }
            Thread.sleep(millis: 200);
        }
    }
}
```

3. **Files:** files are created using the “createFile” function and the content is written in them using the “write” function that uses a “BufferedWriter”. Both the file and the buffered writer are private attributes of the Formula class:

```
/** Crea el archivo del hilo y abre el escritor. */
private void createFile() throws IOException {
    this.archivo = new File("formula_" + id + "_" + rand(min: 1, max: 10000) + ".txt");
    this.archivo.createNewFile();
    this.escriptor = new BufferedWriter(new FileWriter(archivo));
}

/** Escribe una línea en el archivo del hilo. */
private void write(String mensaje) throws IOException {
    escriptor.write(mensaje);
    escriptor.newLine();
    escriptor.flush();    // para poder ver el archivo mientras corre
}
```

Source code:

The code is divided into two files, the Formula class and the Watcher class. In the formula class is implemented the “main” that will execute all the created functions and instructions.

- **Formula.java**

```
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
import java.util.concurrent.ThreadLocalRandom;

/**
 * P = (C + H) * A    donde C, H, A son aleatorios de 1 a 100.
 *
 * Cada hilo:
 * - crea su propio archivo
 * - escribe m mensajes (m aleatorio entre 1 y M)
 * - duerme t milisegundos entre mensaje y mensaje (t aleatorio
entre 10 y T)
 */
public class Formula implements Runnable {

    private final int id;
    private final int maxMensajes;    // M
    private final int maxDormir;      // T
    public double P = 0;
}
```

```
• private File archivo;
• private BufferedWriter escritor;
•
• public Formula(int id, int maxMensajes, int maxDormir) {
•     this.id = id;
•     this.maxMensajes = maxMensajes;
•     this.maxDormir = maxDormir;
• }
•
• /** Aleatorio entre min y max, ambos incluidos. */
• private static int rand(int min, int max) {
•     return ThreadLocalRandom.current().nextInt(min, max + 1);
• }
•
• private int calculateP(int c, int h, int a) {
•     return (c + h) * a;
• }
•
• /** Crea el archivo del hilo y abre el escritor. */
• private void createFile() throws IOException {
•     this.archivo = new File("formula_" + id + "_" + rand(1, 10000)
+ ".txt");
•     this.archivo.createNewFile();
•     this.escriptor = new BufferedWriter(new FileWriter(archivo));
• }
•
• /** Escribe una línea en el archivo del hilo. */
• private void write(String mensaje) throws IOException {
•     escritor.write(mensaje);
•     escritor.newLine();
•     escritor.flush();    // para poder ver el archivo mientras
corre
• }
•
• @Override
• public void run() {
•     try {
•         createFile();
•
•         int m = rand(1, maxMensajes);
•         write("=== Hilo " + id + " | " + m + " mensajes ===");
•
•         for (int i = 1; i <= m; i++) {
•             int c = rand(1, 100);
•             int h = rand(1, 100);
```

```
•         int a = rand(1, 100);
•         int p = calculateP(c, h, a);
•         this.P += p;
•
•         write(String.format("[%d/%d] P = (%d + %d) * %d = %d",
i, m, c, h, a, p));
•
•         int t = rand(10, maxDormir);
•         Thread.sleep(t);
•     }
•
•     double averageP = this.P/m;
•     String apreciacion;
•
•     System.out.println("Hilo " + id + " terminó -> " +
archivo.getName());
•     System.out.println("Promedio de P: " + averageP);
•
•     if(averageP < 5000){
•         apreciacion = "mala";
•     } else if(averageP > 15000){
•         apreciacion = "buena";
•     } else{
•         apreciacion = "promedio";
•     }
•
•     System.out.println("Apreciación: " + apreciacion + ".");
•
•     } catch (IOException e) {
•         System.err.println("Hilo " + id + " error de E/S: " +
e.getMessage());
•     } catch (InterruptedException e) {
•         Thread.currentThread().interrupt();    // buena práctica:
restaurar la bandera
•         System.err.println("Hilo " + id + " interrumpido");
•     } finally {
•         cerrar();
•     }
• }
•
• private void cerrar() {
•     if (escriptor != null) {
•         try {
•             escriptor.close();
•         } catch (IOException ignored) {
```

```
•          // nada que hacer al cerrar
•      }
•  }
•
•  // -----
•
•  public static void main(String[] args) throws InterruptedException
•  {
•      Scanner sc = new Scanner(System.in);
•
•      int n = pedir(sc, "¿Cuántos threads? ", 1);
•      int m = pedir(sc, "¿Cuántos mensajes máximo? ", 1);
•      int t = pedir(sc, "¿Cuánto tiempo dormir máximo (ms)? ", 10);
•
•      List<Thread> hilos = new ArrayList<>();
•      for (int i = 1; i <= n; i++) {
•          Thread hilo = new Thread(new Formula(i, m, t), "formula-"
+ i);
•          hilos.add(hilo);
•          hilo.start();
•      }
•
•      Watcher w = new Watcher(hilos);
•      w.start();
•
•      for (Thread hilo : hilos) {
•          hilo.join();    // esperar a que todos terminen
•      }
•
•      System.out.println("Listo: " + n + " archivos generados.");
•      sc.close();
•  }
•
•  /** Lee un entero válido y mayor o igual al mínimo. */
•  private static int pedir(Scanner sc, String texto, int minimo) {
•      while (true) {
•          System.out.print(texto);
•          if (sc.hasNextInt()) {
•              int valor = sc.nextInt();
•              if (valor >= minimo) {
•                  return valor;
•              }
•              System.out.println("Tiene que ser >= " + minimo);
•          } else {
```

```
•         System.out.println("Eso no es un número.");  
•         sc.next();  
•     }  
•     }  
• }  
• }
```

- **Watcher.java**

```
• import java.util.List;  
•  
• public class Watcher extends Thread{  
•     private List<Thread> threads;  
•  
•     Watcher(List<Thread> ts){  
•         this.threads = ts;  
•     }  
•  
•     @Override  
•     public void run(){  
•         while (true) {  
•             try {  
•                 boolean terminado = true;  
•  
•                 for(Thread hilo : threads){  
•                     String state = hilo.getState().toString();  
•                     String name = hilo.getName();  
•                     System.out.println("Thread: " + name + "; State: "  
• + state);  
•  
•                     if (hilo.getState() != Thread.State.TERMINATED) {  
•                         terminado = false;  
•                     }  
•                 }  
•  
•                 if(terminado){  
•                     System.out.println("Todos los hilos terminados,  
• cerrando watcher");  
•                     break;  
•                 }  
•                 Thread.sleep(200);  
•  
•                 } catch (InterruptedException e) {  
•                     System.out.println(e);  
•                 }  
•     }  
• }
```

Omar Alejandro Sánchez López
Alberto Stephen Dubin Hernández
José Eduardo Moreno Paredes

25/08/2026

- }
- }
- }